

Universidade Federal de Ouro Preto - UFOP
Instituto de Ciências Exatas e Biológicas – ICEB
Departamento de Computação - DECOM
Ciência da Computação

Trabalho Prático I - Sistema de Gerenciamento de Bancos
BCC221 - Programação Orientada a Objetos

Daniel Matos Falcão (25.1.4008)
João Felipe Paiva Paixão (25.1.4014)
Mateus Oliveira Heleno (25.1.4007)
Pedro Henrique Menezes Feo de Castro (25.1.4017)
Pedro Lucca de Castro (25.1.4005)
Rafael Prado Mol e Silva (25.1.4002)

Professor: Guillermo Camara Chavez

Ouro Preto
27 de maio de 2026

1 Introdução

O avanço dos sistemas digitais exige soluções corporativas robustas, escaláveis e seguras para o gerenciamento de fluxos financeiros. O problema abordado neste trabalho consiste na simulação de um ambiente bancário digital dinâmico. Este cenário exige o controle rigoroso de contas correntes e poupanças, autenticação de acessos, histórico de movimentações e ferramentas de crédito. Toda a interação ocorre por meio de uma interface de linha de comando estável, integrada a um sistema automático de persistência de dados.

1.1 Objetivos do Trabalho

O objetivo central deste projeto é aplicar os conceitos teóricos do paradigma de Programação Orientada a Objetos (POO) em C++. O sistema foi projetado para consolidar os seguintes pilares:

- **Encapsulamento:** Proteção de atributos críticos através de métodos seletores e modificadores.
- **Herança e Polimorfismo:** Especialização de classes e redefinição de métodos em tempo de execução.
- **Associação e Composição:** Vinculação estrutural entre entidades de forma eficiente em memória.
- **Persistência e STL:** Manipulação de fluxos de arquivos (CSV) e uso de coleções genéricas da Standard Template Library.

1.2 Escopo do Sistema

- Cadastro e listagem de perfis (Clientes e Gerentes).
- Mecanismo de transações financeiras validadas (Saques, Depósitos e Transferências) com histórico em extrato.
- Cálculo automatizado de rendimento para contas poupança.
- Vínculo e controle de carteira de clientes por parte dos gerentes.
- **Escopo Adicional:** Um módulo avançado de Cartão de Crédito que suporta compras parceladas, faturamento mensal, controle dinâmico de limites e bloqueio de segurança.

2 Descrição do Sistema

Nesta seção, vamos detalhar o funcionamento prático do programa, como as entidades são manipuladas e como o usuário interage com o menu textual através do terminal.

2.1 Entidades Manipuladas e Funcionalidades

O sistema simula com precisão as operações essenciais de uma agência bancária digital, gerenciando as seguintes entidades centrais:

- **Clientes:** Representam os usuários finais do banco. O sistema armazena seus dados cadastrais (nome, trabalho), credenciais de acesso (login, senha) e informações financeiras específicas, como a remuneração, o tipo de conta (Corrente ou Poupança), o saldo em conta e um vetor dinâmico que armazena seu histórico de transações (extrato).
- **Gerentes:** Atuam como administradores do sistema. Eles possuem credenciais de acesso e são responsáveis por gerenciar uma carteira de clientes vinculados a eles.
- **Transações:** Objetos gerados dinamicamente a cada movimentação financeira (Saque, Depósito ou Transferência). Elas registram o valor, a data, o horário e armazenam ponteiros para os clientes envolvidos na operação.
- **Módulo de Crédito (Cartão de Crédito):** Uma subestrutura acoplada ao cliente que permite a criação de um cartão com limite baseado na sua remuneração. Controla compras parceladas, faturas abertas e bloqueios de segurança.

2.1.1 Interação com o Usuário via Terminal

A interação com o sistema é feita inteiramente por linha de comando através de uma interface CLI (Command Line Interface) intuitiva e robusta. O programa utiliza uma lógica de loop (*do-while*) no arquivo *main.cpp* para manter o menu ativo até que a opção de encerramento seja acionada. Para garantir uma boa experiência de usuário e evitar falhas de execução, foram implementadas as seguintes rotinas de controle de entrada:

- **Fluxo de Cadastro Dinâmico:** Ao escolher as opções de cadastro (1 ou 2), o sistema entra em modo de leitura. O utilizador é guiado passo a passo por perguntas sucessivas no terminal ("Nome: ", "Login: ", "Senha: "). Graças ao uso de *std::getline*, o utilizador pode digitar nomes completos com espaços perfeitamente. No cadastro do cliente, há uma validação em tempo real, se for escolhida uma conta do tipo "Poupança", o sistema calcula e exibe imediatamente na tela a taxa de rendimento mensal correspondente baseada na remuneração informada.
- **A Mecânica de Transações (Opção 3):** Quando o utilizador solicita a criação de uma transação, o terminal pede o tipo (Transferência, Depósito ou Saque). O programa faz um tratamento inteligente de strings para aceitar termos em maiúsculas ou minúsculas. Se a operação envolver movimentação de saída (Saque ou Transferência), o sistema solicita os nomes dos envolvidos, busca os respectivos ponteiros na memória e valida o saldo atual do cliente. Caso o saldo seja insuficiente, o sistema bloqueia o processo na hora e emite um aviso claro na tela, impedindo furos nas contas.
- **Associação Administrativa (Opção 5):** O utilizador consegue vincular clientes a gerentes digitando os nomes de ambos. O sistema faz a busca no banco de dados em memória e estabelece a relação através de ponteiros nativos, atualizando a carteira de clientes daquele gerente de forma imediata.

- **Visualização de Dados e Extratos (Opções 4, 6 e 7):** Ao listar ou buscar um cliente/gerente, as informações não aparecem de forma compacta ou confusa. O terminal renderiza um bloco estruturado e tabulado com todos os dados protegidos da entidade (exibindo o nome, trabalho, saldo e o histórico completo de transações associadas com data e hora).
- **Submenu de Cartão de Crédito (Opção 8):** Ao selecionar esta opção, o loop principal do banco é pausado, e o usuário é transferido para um ecossistema de autoatendimento exclusivo de crédito.
- **Salvamento Automático e Encerramento (Opção 9):** Esta é a opção que garante a segurança de todo o sistema. Em vez de simplesmente fechar o programa e perder os dados armazenados na memória RAM, a opção 9 aciona os motores de persistência. O programa exibe uma mensagem de alerta ("Salvando registros nos arquivos..."), aguarda um segundo para dar feedback visual ao usuário e chama as funções que convertem os vetores de objetos de volta para o formato plano delimitado por vírgulas. Os arquivos *clientes.csv*, *gerentes.csv*, *transacoes.csv*, *cartoes.csv* e *compras.csv* são sobrescritos com as informações atualizadas. Após a gravação completa no disco, o loop *do-while* é encerrado, o programa limpa os ponteiros alocados dinamicamente para evitar vazamento de memória e finaliza a execução com segurança.
- **Navegação e Confirmação Visual:** Para evitar que as informações desapareçam rapidamente após uma listagem ou transação, o sistema congela a tela temporariamente com a mensagem "Pressione Enter para voltar ao menu!". O terminal só é limpo pela função *limparTerminal()* após a confirmação do utilizador, dando-lhe total controle sobre o tempo de leitura dos dados na tela.

3 Decisões de Projeto

3.1 Abstração e Encapsulamento Rígido

O encapsulamento foi tratado como prioridade para garantir a integridade dos dados financeiros e cadastrais do sistema:

- **Atributos Privados e Protegidos:** Dados sensíveis, como o saldo, a taxa de rendimento e as senhas de acesso, foram declarados como *private* ou *protected* nas classes *Pessoa* e *Cliente*. Isso impede que modificações externas indevidas ocorram de forma direta fora do escopo das classes.
- **Métodos Seletores e Modificadores (*Getters* e *Setters*):** O acesso a esses atributos é feito exclusivamente por meio de interfaces públicas bem definidas (como *getSaldo()* e *setSaldo()*). Esses métodos contêm validações internas explícitas, por exemplo, a função *setRemuneracao(double novaRemuneracao)* no arquivo *cliente.cpp* rejeita imediatamente valores negativos, exibindo uma mensagem de erro e protegendo a consistência do objeto.

3.2 Herança e Polimorfismo Dinâmico

A arquitetura do sistema utiliza herança e polimorfismo para evitar a duplicação de código e permitir a extensibilidade do software:

- **Classe Base Abstrata (*Pessoa*):** Atua como a raiz da hierarquia de usuários, concentrando atributos comuns como *nome*, *trabalho*, *login* e *senha*. Ela define o método virtual puro *virtual string getHeader() = 0;*, tornando-se uma interface que obriga as classes derivadas a fornecerem suas próprias estruturas de cabeçalho para salvamento.
- **Especialização (*Cliente* e *Gerente*):** Reutilizam toda a estrutura base de *Pessoa* por meio de herança pública (*class Cliente : public Pessoa*) e adicionam seus comportamentos específicos.
- **Sobrescrita de Métodos (Override):** O método *virtual void exibirDados();* foi implementado na classe mãe e sobrescrito nas classes filhas. Em tempo de execução, quando o sistema interage com ponteiros para *Pessoa*, o mecanismo de tabelas virtuais (vtable) garante que o comportamento específico de cada perfil seja chamado (Polimorfismo Dinâmico).

3.3 Associação por Ponteiros e Composição Estrita

O gerenciamento dos relacionamentos entre os objetos em memória foi desenhado para balancear o desempenho com a consistência de dados:

- **Associação por Ponteiros (Agregação):** No arquivo *gerente.hpp*, a carteira de clientes é modelada como um vetor dinâmico de ponteiros nativos (*vector<Cliente*> clientes;*). Da mesma forma, a classe *Transacao* armazena ponteiros para as partes envolvidas. Essa abordagem evita a cópia desnecessária de objetos pesados na memória RAM e garante o princípio da consistência única, se o saldo de um cliente for alterado dentro de uma transação, essa mudança reflete instantaneamente quando o gerente listar sua carteira, pois ambos apontam para o mesmo endereço de memória.
- **Composição Estrita:** O relacionamento entre *Cliente* e *CartaoCredito* foi modelado através de composição (um objeto real *CartaoCredito cartao;* embutido nos atributos privados do cliente). Isso amarra rigidamente o ciclo de vida do cartão ao do seu titular, se o cliente deixar de existir na memória, o cartão e seu histórico de compras parceladas são destruídos simultaneamente.

3.4 Utilização da Biblioteca STL

A Standard Template Library (STL) foi explorada extensivamente para fornecer robustez e agilidade ao código:

- ***std::vector*:** Utilizado como contêiner dinâmico principal para gerenciar os cadastros (*vector<Cliente>*, *vector<Gerente>*) e os históricos devido ao seu acesso indexado rápido em memória contígua e gerenciamento de capacidade automático.

- ***std::stringstream***: Peça fundamental no motor de leitura dos arquivos, permitindo fatiar cada linha de texto extraída do arquivo CSV usando o caractere delimitador vírgula (,) como critério de corte através de *std::getline(ss, campo, ',')*.
- **Programação Genérica (Templates)**: Para otimizar e modularizar operações comuns, foram criadas funções baseadas em templates no arquivo *auxiliares.hpp*. A função *template<typename T> void escreverCSV(...)* é capaz de salvar tanto vetores de clientes quanto de gerentes de forma genérica, reduzindo drasticamente as linhas de código redundantes.

3.5 Persistência de Dados (Lógica de Arquivos)

O sistema implementa persistência de dados de forma automatizada e transparente no formato plano CSV (Comma-Separated Values):

- **Carregamento Automático**: Ao iniciar o programa, o *main()* invoca rotinas de leitura (*carregarClientes*, *carregarGerentes*, *carregarTransacoes* e *carregarCartoes*) que abrem os arquivos com *std::ifstream*, processam os tokens e reconstroem os objetos na memória RAM.
- **Escrita e Persistência**: Ao selecionar a opção de saída, o sistema abre fluxos de saída com *std::ofstream* sobre os mesmos arquivos, sobrescrevendo-os com os cabeçalhos oficiais retornados por *getHeader()*. Na sequência, a sobrecarga do operador de inserção (*friend ostream& operator<<*) extrai os atributos internos formatados exatamente com delimitadores de vírgula, preservando o estado do sistema para as próximas execuções.

4 Arquitetura do Projeto

O sistema foi desenvolvido seguindo o princípio da modularização, separando as interfaces e protótipos de funções (arquivos de cabeçalho *.hpp*) das suas respectivas ligações lógicas e implementações (arquivos *.cpp*). Essa divisão garante a legibilidade, facilita a manutenção do código e otimiza o tempo de compilação do projeto.

4.1 Divisão dos Módulos e suas Responsabilidades

- ***pessoa.hpp* / *pessoa.cpp***: Centraliza a classe base abstrata *Pessoa*. Ela é responsável por gerenciar os dados cadastrais básicos comuns a todos os usuários (nome, trabalho, login e senha) e define o contrato polimórfico através do método virtual puro *getHeader()*.
- ***cliente.hpp* / *cliente.cpp***: Contém a classe *Cliente*, que herda publicamente de *Pessoa*. É responsável pelo controle de atributos financeiros específicos (remuneração, tipo de conta, saldo e taxa de rendimento), além de gerenciar a coleção de ponteiros para o extrato de transações e a composição com o cartão de crédito.

- ***transacao.hpp* / *transacao.cpp***: Define o motor de movimentações financeiras. Essa classe gerencia os dados individuais de cada operação (tipo, valor, data e horário) e armazena ponteiros para os clientes envolvidos para aplicar as deduções ou acréscimos de saldo em tempo real.
- ***credito.hpp* / *credito.cpp***: Módulo avançado responsável pelo gerenciamento de crédito. Implementa as classes *CartaoCredito* e *CompraParcelada*, lidando com a lógica de concessão de limites, bloqueios de segurança, registro de parcelas e quitação de faturas.
- ***auxiliares.hpp* / *auxiliares.cpp***: Concentra funções utilitárias globais do sistema, como rotinas de limpeza de terminal, tratamento de buffers do teclado, funções genéricas baseadas em templates para busca de elementos e algoritmos para leitura e escrita dos arquivos CSV.

4.2 Funcionamento do Fluxo Principal (*main.cpp*)

O arquivo *main.cpp* funciona como o cérebro orquestrador da aplicação. O fluxo de execução do programa segue três etapas rígidas e sequenciais:

- **Inicialização e Carga**: Assim que o programa inicia, os vetores principais (*clientes*, *gerentes* e *transacoes*) são declarados e é feita a pré-alocação de memória com *reserve()*. Logo em seguida, as funções do módulo *auxiliares* abrem os arquivos em disco e restauram o estado completo do sistema para a memória RAM.
- **Loop de Execução Interativo**: O programa entra em uma estrutura de repetição contínua (*do-while*) controlada pelo menu textual. A cada escolha do usuário, o terminal limpa a tela e direciona o fluxo para a função correspondente (cadastro, listagem ou criação de transações). Caso o usuário entre no menu de cartões (Opção 8), o controle é transferido temporariamente para o loop do arquivo *credito.cpp*.
- **Persistência e Desalocação**: Quando a opção 9 (Sair) é acionada, o loop principal é quebrado. O programa realiza a gravação em massa de todos os dados atualizados de volta para os respectivos arquivos CSV. Por fim, um loop *for* varre o vetor de transações executando o operador *delete* para limpar os blocos de memória alocados dinamicamente, prevenindo qualquer vazamento (memory leak) antes de encerrar o processo.

5 Diagramas UML

Para mapear e validar a arquitetura lógica do sistema de gerenciamento bancário, foi desenvolvido um Diagrama de Classes utilizando a notação padrão UML (*Unified Modeling Language*). O modelo reflete visualmente a distribuição de responsabilidades, os níveis de encapsulamento e os acoplamentos entre as entidades do domínio do software.

5.1 Especificação Estrutural das Classes

O modelo estático é composto pelas seguintes classes e componentes principais:

- **Pessoa (Classe Base Abstrata):**

- Atributos Protegidos/Privados: nome e trabalho (protected), login e senha (private).
- Métodos Principais: *exibirDados()* (virtual), métodos modificadores/seletores (*getters/setters*) para os atributos identitários e o método virtual puro *getHeader()*.

- **Cliente (Classe Especializada):**

- Atributos Privados: remuneracao, saldo, tipoDeConta, taxaDeRendimento, temCartao (booleano), a subestrutura cartao e o vetor de histórico transacoes.
- Métodos Principais: Sobrescrita (override) de *exibirDados()* e *getHeader()*, além de rotinas de manipulação financeira como *getSaldo()*, *setSaldo()*, *criarCartao()* e *setTransacao()*.

- **Gerente (Classe Especializada):**

- Atributos Privados: Coleção dinâmica *clientes* (do tipo *vector<Cliente*>*).
- Métodos Principais: Sobrescrita de *exibirDados()* e *getHeader()*, além do método de vinculação administrativa *setCliente()* e seletor *getClientes()*.

- **Transacao (Classe de Operação):**

- Atributos Privados: valor, tipo, data, horario e a coleção clientesEnvolvidos (*vector<Cliente*>*).
- Métodos Principais: Métodos de acesso para metadados (*getTipo*, *getValor*, *getData*) e funções globais de processamento operacional associadas (*processarTransferencia*, *processarSaque*, *processarDeposito*).

- **CartaoCredito e CompraParcelada (Módulo de Crédito):**

- Atributos Privados: limiteTotal, limiteDisponivel, valorFatura, bloqueado, faturaGerada e o histórico comprasParceladas (em *CartaoCredito*), descricao, valorTotal, quantidadeParcelas, valorParcela e parcelasPagas (em *CompraParcelada*).
- Métodos Principais: *realizarCompraParcelada()*, *pagarFatura()*, *gerarFatura()*, *bloquear()* e *desbloquear()*.

5.2 Mapeamento dos Relacionamentos Semânticos

Conforme ilustrado no diagrama anexado, as interações entre as classes seguem as regras estritas da orientação a objetos:

- **Herança (Seta com triângulo preenchido/vazio apontando para a base):** Conecta `Cliente` $\rightarrow$ `Pessoa` e `Gerente` $\rightarrow$ `Pessoa`, indicando a especialização e reutilização de código.
- **Agregação/Associação Direcionada (Seta simples ou losango aberto):**
 - Relação de 1 para muitos ($1 \rightarrow *$) de `Gerente` para `Cliente`, mapeando a carteira onde um gerente gerencia múltiplos clientes via ponteiros.
 - Relação de muitos para muitos ($* \rightarrow *$) entre `Transacao` e `Cliente`, onde uma transação aponta para os clientes envolvidos e cada cliente mantém referências das suas transações no extrato.
- **Composição (Losango preenchido):** Vinculação forte existente de `Cliente` para `CartaoCredito`, explicitando que o cartão é parte integrante do ciclo de vida da conta do cliente. Uma relação de composição interna semelhante ocorre de `CartaoCredito` para `CompraParcelada`.

5.3 Visualização do Diagrama de Classes

A Figura 1 apresenta o diagrama de classes UML completo que serviu de especificação para a codificação das estruturas do sistema.



Figura 1: Diagrama de Classes UML do Sistema Bancário e Módulo de Crédito.

6 Testes Realizados

Esta seção detalha os cenários de testes manuais executados para homologação do sistema bancário, descrevendo a quantidade de dados validados, o tratamento de exceções de entrada e a resolução de inconsistências operacionais.

6.1 Cenários de Testes e Dados Consolidados

Com base na execução dos testes e na gravação automática dos arquivos CSV anexados ao sistema, a quantidade total de dados testados engloba as seguintes entidades reais:

- **Cientes Cadastrados:** Foram validados os perfis de 8 clientes distintos no arquivo *clientes.csv* (Pedro Henrique, Pedro Lucca, Daniel, Mateus, Joao, Rafael, Marcos e

falcao). Os cenários cobriram a abertura tanto de contas do tipo "poupança"(com cálculo automático de rendimento mensal baseado em 5% da remuneração) quanto do tipo "corrente"(com taxa de rendimento nula).

- **Gerentes Cadastrados:** Foram inseridos e validados 2 perfis administrativos no arquivo *gerentes.csv* (Jadson e mvzadas), cobrindo o fluxo de associação e controle das carteiras de clientes.
- **Transações Financeiras:** Foram executadas e auditadas 4 movimentações financeiras registradas em *transacoes.csv* (sendo 3 depósitos nos valores de R\$ 100, R\$ 100 e R\$ 5000, e 1 saque no valor de R\$ 100), validando a atualização instantânea de saldos em memória e em arquivo.
- **Módulo de Crédito (Cartões e Compras):** Foram gerados registros de cartões para todos os 8 clientes em *cartoes.csv*, dos quais 2 portavam o cartão ativo (*temCartao = 1*), o cliente Pedro Henrique e o cliente falcao. No arquivo *compras.csv*, foi homologada com sucesso 1 compra parcelada de "Monitor" no valor total de R\$ 1000, dividida em 5 parcelas, das quais 1 já consta como liquidada.

6.2 Tratamento de Erros e Validações de Fluxo

Para garantir o funcionamento adequado do programa, evitar warnings e impedir o corrompimento do estado financeiro do sistema, foram implementadas rotinas estritas de tratamento de exceções e erros de entrada:

- **Blindagem contra Entradas Inválidas (Inputs Alfanuméricos):** Na função *lerValor(int n)* do arquivo *auxiliares.cpp*, o recebimento de opções do menu foi envelopado em um bloco *try-catch* utilizando *std::stoi*. Caso o usuário digite uma sequência contendo caracteres de texto (ex: "1aa"ou "abc"), o método captura a exceção *invalid_argument*, impede a quebra do fluxo operacional e força nova digitação limpa dentro do intervalo permitido.
- **Restrição de Valores Negativos ou Nulos:** Durante as operações de depósito, saque, transferência ou definição de remuneração, o sistema valida ativamente se o montante fornecido é maior que zero. Caso um valor inválido ou negativo seja inserido, o fluxo limpa a flag de erro via *cin.clear()*, descarta o lixo residual com *limparBuffer()* e exige nova entrada, impossibilitando saldos inconsistentes.
- **Validação de Formatos de Data e Horário:** O motor de transações exige a inserção estrita do padrão de tamanho e caracteres delimitadores. A inserção da data é submetida ao laço que checa se a string possui tamanho idêntico a 10 e se as barras ocupam as posições corretas (*data[2] == '/'* e *data[5] == '/'*). O mesmo rigor é aplicado ao horário (*horario.length() == 5* e *horario[2] == ':'*), prevenindo a escrita de metadados corrompidos nos arquivos CSV.
- **Controle de Saldo Insuficiente e Bloqueio de Cartão:** As rotinas de verificação de saldo e de compras parceladas validam as pré-condições matemáticas antes de efetivar

qualquer débito. Se o saldo ou o limite disponível for menor que o valor pretendido, a operação é abortada imediatamente com avisos textuais claros na interface do usuário. Além disso, compras parceladas em cartões bloqueados são rejeitadas de imediato, garantindo conformidade com os requisitos de segurança.

7 Conclusão

7.1 Aprendizado Obtido

A implementação deste sistema de gerenciamento bancário foi fundamental para consolidar de forma prática e profunda os conceitos teóricos de Programação Orientada a Objetos (POO) em C++. O maior aprendizado foi compreender a importância de uma modelagem arquitetural sólida, como as decisões tomadas nos diagramas UML refletem diretamente na facilidade e eficiência da codificação. A aplicação real de conceitos como herança (centralizando atributos comuns na classe base abstrata *Pessoa* e estendendo-os de forma pública para *Cliente* e *Gerente*) e polimorfismo (através da sobrescrita de métodos virtuais como *exibirDados* e *getHeader*) demonstrou o poder da reutilização de código. Além disso, o uso estratégico de ponteiros nativos para estabelecer relacionamentos de associação e agregação simulou com fidelidade um domínio de regras de negócios complexo, permitindo gerenciar a memória sem duplicação desnecessária de instâncias.

7.2 Dificuldades Encontradas

O principal desafio técnico do projeto residiu no mecanismo de persistência cruzada de dados e sincronização via arquivos CSV. Como o sistema utiliza referências e ponteiros para ligar gerentes a clientes, e transações aos seus respectivos participantes, ler puramente as strings de texto plano dos arquivos não era suficiente. Foi necessário desenvolver algoritmos inteligentes de busca em memória (através de templates genéricos criados em *auxiliares.hpp*) que localizam as instâncias pelos nomes em tempo de inicialização para reconectar os ponteiros e restaurar o estado exato do sistema. Outro obstáculo puramente prático foi lidar com a manipulação de fluxos de entrada (*std::cin*) no terminal. A mistura de leituras numéricas com a captura de strings compostas contendo espaços (como nomes inteiros de clientes) frequentemente gerava comportamentos inesperados causados pelo lixo residual nos buffers. Esta dificuldade foi mitigada com o isolamento de funções utilitárias como *limparBuffer()* e o tratamento de exceções com *try-catch* na conversão de tipos de dados.

7.3 Melhorias Futuras

Embora o software atenda com sucesso a todos os requisitos obrigatórios e entregue um módulo bônus robusto de crédito parcelado, algumas evoluções poderiam ser implementadas numa versão futura:

- **Ponteiros Inteligentes:** Substituir o uso de ponteiros nativos por smart pointers (*std::shared_ptr* e *std::weak_ptr*) da biblioteca STL para automatizar a gerência de

ciclo de vida dos objetos e eliminar a necessidade de varreduras manuais com o operador *delete*, extinguindo qualquer risco residual de vazamento de memória.

- **Interface Gráfica (GUI):** Evoluir o sistema para além da linha de comandos do terminal, acoplando uma interface visual, seria uma grande melhoria para nosso projeto.